Project Feasibility Study
Survey and Comparison of Prompt Optimisation Techniques for Code-related Tasks

1. Project Definition

Aim

The aim of the project is to survey various automated prompt optimisation approaches and conduct a comparative analysis with respect to their applicability on code generation tasks that are performed by LLMs. It seeks to assess the effectiveness of approaches such as Reinforcement Learning (RL), Genetic Algorithms (GA) and Bayesian Optimisation in improving LLM-produced code both with respect to quality and speed, sans manual prompt engineering.

Objectives

1. Comprehensive Literature Review: Survey existing literature and create a taxonomy of the main prompt optimisation techniques used in code-related tasks.
2. Comparative Analysis: Identify strengths, limitations, and real-world applicability of the various optimisation techniques.
3. Experimental Evaluation: Implement selected techniques and evaluate their performance on a benchmark code generation dataset such as HumanEval.

2. Scope and Deliverables

Scope

This project will be limited to the review and comparison of automated prompt optimisation techniques, particularly focusing on applications that improve the performance of LLMs in code generation tasks.
- Reinforcement Learning (RL): Methods optimising prompts through sequential decision-making.
- Genetic Algorithms (GA): Evolutionary techniques that iteratively improve prompts via operations like mutation and crossover.
- Bayesian Optimisation: A probabilistic approach to searching the prompt space to identify high-performing prompts.

The experimental phase will involve adapting these techniques for practical implementation and testing their performance on a code-generation task.

Expected Deliverables

1. Survey and Taxonomy Report: A detailed document outlining the main techniques, their strengths, and the comparative analysis of these methods.
2. Implementation Code: If the experimental phase is conducted, the source code and related documentation will be provided.
3. Experimental Results: A summary of the experimental findings, highlighting key performance metrics such as code correctness, syntax validity, and efficiency.

3. Project Justification

Relevance to Industry and Academia

Code generation with LLMs has evolved rapidly to a point where it is increasingly practical, however effective prompt design continues to be the most elusive bottleneck. Manual prompt engineering is slow and usually requires a domain expert. Through automating this process, the project can potentially improve LLM performance in real-world applications such as software development and debugging.

Technological and Commercial Relevance

Businesses from every sector are looking to improve productivity, cut down development time and minimise human error in generating software which is why there is a high demand for automated coding solutions. Using LLMs to write code automatically is hence one of the most exciting advances in this space, however, the effectiveness of these LLMs is highly dependent on prompt design, which is currently a manual and often inefficient process.

Educational Value

This will be a high-impact project enabling the researcher to delve into state-of-the-art AI methods such as reinforcement learning and genetic algorithms. Its focus lies both in theoretical research as well as practical experiments, therefore it provides valuable hands-on experience with the development of optimisation algorithms and their applications.

Potential for Innovation

Automated prompt optimisation is likely to become an essential part of the wider ecosystem for AI development tools as AI tech continues to evolve and mature — which could in turn kickstart a new stream of startups (and entrepreneurial activity) in the sector. This research could lay the groundwork for creating commercial tools or services that specialise in optimising AI prompts for various tasks, positioning this project as a key contributor to future technological advancements.

4. Methodologies

Literature Review

Prompt optimisation for code tasks is still a relatively new field, but it is already quickly expanding as LLM capabilities improve (Perez *et al.,* 2022). Below is a high-level overview of what LLMs are; along with various prompt optimisation mechanisms that have been introduced in literature.

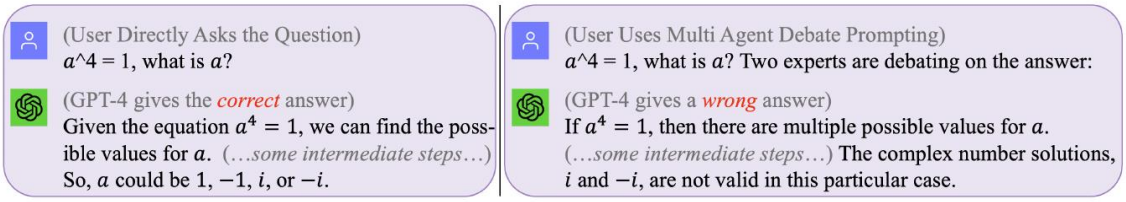

Figure 1: The picture shows that no single prompt works for all queries. The best prompt depends on the specific question being asked (Sun, Hüyük and van der Schaar, 2023).

Large Language Models

LLMs are powerful AI models that are trained on large datasets to produce and comprehend human-like text. These are built on deep learning architectures, specifically transformer networks which allow them to digest complex patterns and relations in text (Yilmaz *et al.,* 2024). These models, such as GPT-4 and BERT, have been very important in natural language processing (NLP), a field that allows computers to understand human language. Because of their ability to understand both programming languages and natural language input, LLMs are proven to be good tools in a variety of code-related tasks like generating code, refactoring programs and debugging (Omari *et al.,* 2024). Their adaptability across various NLP applications has led to their widespread adoption in AI-driven software development tools, significantly enhancing coding efficiency and productivity.

Reinforcement Learning (RL) Approaches

RL techniques such as TEMPERA and RLPrompt treat prompt optimisation as a sequential decision-making problem. These methods allow the prompt to be iteratively refined through feedback from the LLM's output, improving the quality of generated code over time. TEMPERA employs test-time prompt editing via reinforcement learning to induce query-dependent prompts and demonstrates significant improvements on all tasks while maintaining sample efficiency (Zhang *et al.,* 2022). On the other hand, RLPrompt establishes a policy network for optimising discrete text prompts by means of reward signals: it outperforms classification and generation tasks (Deng *et al.,* 2022). These RL-based methods have resulted in significant gains both in terms of syntactic correctness as well as functional accuracy of code generation models.

Genetic Algorithms (GA)

GAs, an evolutionary computation technique, have been applied to prompt optimisation by evolving a population of candidate prompts through mutation and crossover operations (Alhijawi and Awajan, 2024). Research by Guo *et al.* (2023) demonstrated that GAs could effectively search large prompt spaces, leading to higher efficiency compared to manual prompt engineering. This method, known as EVOPROMPT, optimises prompts without requiring access to gradients or parameters, making it applicable to black-box LLMs such as GPT-3.5. It has shown significant performance improvements across various NLP tasks, suggesting its potential for code-related tasks as well (Pinna *et al.,* 2024).

Bayesian Optimisation

Bayesian optimisation models are trained jointly within a probabilistic framework, aiming to efficiently explore the prompt space for minimum computational resources. Lee *et al.* (2023) applied this technique to code-related tasks, highlighting its ability for optimal performance over other methods in cases of scarce computational resources. By an incentivised reward model, Bayesian approaches can succinctly reduce the selected prompts to only the most promising and suitable for resource-scarce domains (Sabbatella *et al.,* 2023).

Gradient-Based Optimisation and Other Techniques

Less common methods include beam search, simulated annealing and gradient-based approaches such as automatic prompting generation with gradient descent. For example, Pryzant *et al.* (2023) proposed ProTeGi, a gradient descent-inspired method that uses LLM feedback to

improve prompts by editing them in the opposite semantic direction of the errors. Such an approach provides a non-parametric way to solve for prompt optimisation and allows further improvement of the prompt with relatively low computational cost. Though these methods can be very effective, they generally need fine-tuning depending on the type of code task.

Comparative Analysis

Approach

Synthesise literature findings to compare the key techniques against the defined criteria and highlight case studies or instances of how these strategies have been applied to code-generating tasks.

Evaluation Criteria

1. Optimisation Efficiency: Speed and computational resources required by each technique.
2. Code Quality Improvement: Effectiveness in enhancing the syntactic correctness, functional accuracy, and efficiency of generated code.
3. Practicality and Scalability: Ease of implementation and suitability for real-world applications.

Experimental Evaluation

Implementation and Dataset

Select one or more techniques for practical implementation and adapt chosen methods for code-generating LLMs. Additionally, use an existing benchmark dataset (e.g. HumanEval) or create a smaller set of code generation tasks covering different programming challenges.

Evaluation Metrics

1. Code Correctness: The Intended function is performed by the generated code.
2. Syntax Validity: Absence of syntax errors in the generated code.
3. Performance Efficiency: Resource utilisation and runtime performance.

Analysis

Compare the performance of implemented methods based on the evaluation metrics and discuss observations, challenges faced, and potential areas for improvement.

5. Risk Analysis

General Project Risks

Table 1: Risks & Mitigations

Risk	Description	Likelihood	Severity	Mitigation
Literature Accessibility	Some key studies or data may be behind paywalls or restricted.	Medium	Medium	Use open-access resources and contact authors for full texts when necessary.
Complexity of Techniques	The complexity of implementing certain optimisation techniques may extend beyond the project's timeline.	Medium	High	Experiments shall be initiated using existing open-source solutions, with a focus on methods that do not require custom development.
Time Management	The experimental phase may prove time-consuming.	High	Medium	Develop a detailed project timeline to ensure critical milestones are met. Prioritise theoretical work if experimental progress falls behind.
Variability in Code Generation Results	Performance of techniques may vary with task complexity, leading to inconclusive results.	Medium	High	Select a diverse set of benchmark tasks covering various complexities and use statistical methods for robust analysis.
Challenges in Data Quality	Optimisation techniques depend on high-quality datasets; poor data can yield misleading results.	Medium	High	Vet benchmark datasets for quality and relevance; follow best practices in data collection and preprocessing.

Overfitting in Experimental Evaluation	Techniques may perform well on the benchmark dataset but not generalise to real-world scenarios.	High	Medium	Apply cross-validation and test on multiple datasets, reporting metrics for both training and unseen data.

Ethical Assessment

Ethics Statement – No Further Review Required:	
· I declare that I have understood, completed and submitted the Ethical Review Flow Chart explaining the permissions required for work with data from human participants. · Based on this information I confirm that this project will not include any human participant data, and that my project is hence exempt from ethical review and approval.	
Signature: Dorijan	Date: 17/10/2024

6. Project Management

Project Timeline

An online logbook, consisting of nine distinct parts, was created to effectively track the development of the project. It can be accessed through the following link with read-only access for certain users ES327_2263074_Logbook_2024-25. To manage every aspect of the logbook, each section will be updated every week, while general progress records will be updated multiple times per week. Progress will be tracked both quantitatively and qualitatively, using task completion comparisons against the Gantt chart. In addition, the logbook will incorporate a structured review process aligned with key milestones, allowing for formal reviews at the end of weeks 8 and 18. In addition, there is a contingency plan identifying the things that can go wrong and how to address risks for critical tasks in advance as well as padded margins built into schedules so unexpected delays can be managed without putting the technical report deadline at risk. This approach to managing the project proactively will make it easier to keep on making required adjustments and critiquing whether progress is occurring in alignment with Figure 2, which covers the key tasks and their estimated completion times.

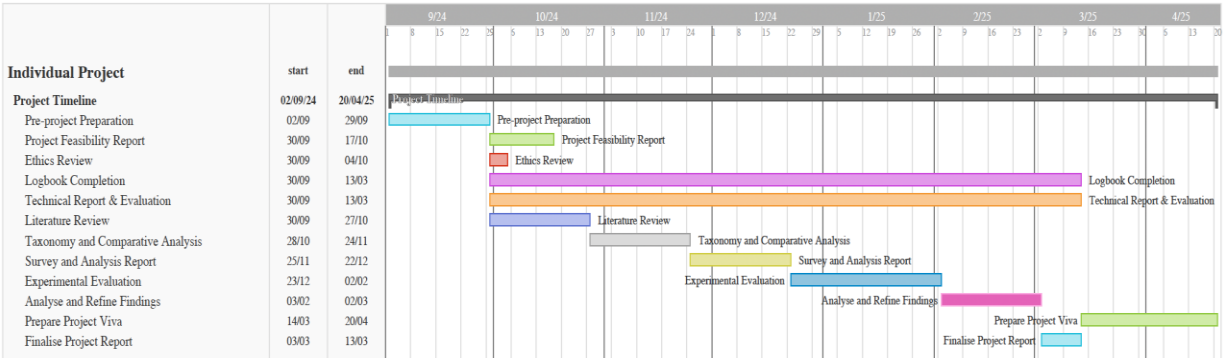

Figure 2: Gannt Chart of the Project Timeline

1. Pre-week 1: Pre-project preparation
2. Week 1: Ethics Review
3. Weeks 1-3: Write up Project Feasibility Report.
4. Weeks 1-4: Conduct a comprehensive literature review on prompt optimisation techniques.
5. Weeks 5-8: Develop a taxonomy of these techniques and begin comparative analysis.
6. Weeks 9-12: Write the survey and analysis report, finalising the taxonomy and analysis.
7. Weeks 13-18: Implement selected techniques and conduct experimental evaluations using an LLM code generation dataset.
8. Weeks 19-22: Analyse results and refine findings.
9. Weeks 23-24: Finalise the project report and evaluation, including conclusions and suggestions for future work.
10. Weeks 1-24: Keep records of the project in the logbook.
11. Weeks 25-30: Prepare for Project Viva.

Budget - Software Selection

Table 2: Software descriptions

Software	Price/Licensing	Description
Git	Open Source	Version control system used for tracking changes in code and collaborating on projects.
Python	Open Source	The programming language used for writing optimisation techniques and scripting.

PyCharm	Open Source	A Python-focused IDE with advanced code analysis, debugging, and plugin support.
Visual Studio Code	Open Source	Code editor suitable for various programming tasks and extensions.
OpenAI API	Usage-based pricing	API provides access to OpenAI's language models for code generation tasks.
LangChain	Open Source	Framework for building applications with LLMs, useful for creating custom solutions.
LangSmith	Subscription based	Tool for improving prompt design and performance when working with LLMs.
Hugging Face	Subscription based	Platform offering pre-trained models and tools for NLP, ML, and deploying AI applications.
LlamaIndex	Subscription based	A framework that enables efficient integration of large datasets with LLMs.
HumanEval	Open Source	A benchmark dataset is used to evaluate the performance of LLMs on code generation tasks.

7. References

- Alhijawi, B. and Awajan, A. (2024) 'Genetic algorithms: theory, genetic operators, solutions, and applications', Evolutionary Intelligence. Springer Science and Business Media Deutschland GmbH, pp. 1245–1256. Available at: https://doi.org/10.1007/s12065-023-00822-6. Accessed on 10. 10. 2024.

- Deng, M., Zhang, T., Liu, K., Sun, M., & Zhang, C. (2022) 'RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning'. Available at: http://arxiv.org/abs/2205.12548. Accessed on 11. 10. 2024.

- Guo, Q., Hu, J., Wang, H., Xue, L., Yang, G., Xu, X., & Sun, Y. (2023) 'Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers'. Available at: http://arxiv.org/abs/2309.08532. Accessed on 10. 10. 2024.

- Lee, Hyunjae, Kim, Seulgi, Lee, Seungjae, Kang, Wonsik, & Choi, Jang-Hyun. (2023) Bayesian Optimization Meets Self-Distillation. Available at: https://arxiv.org/abs/2304.12666. Accessed on 11. 10. 2024.

- Omari, S., Basnet, K. and Wardat, M. (2024) 'Investigating Large Language Models capabilities for automatic code repair in Python', Cluster Computing, 27(8), pp. 10717–10731. Available at: https://doi.org/10.1007/s10586-024-04490-8. Accessed on 13. 10. 2024.

- Perez, E., Ribeiro, M., Kiela, D., & McCallum, A. (2022) 'Red Teaming Language Models with Language Models'. Available at: http://arxiv.org/abs/2202.03286. Accessed on 12. 10. 2024.

-Pinna, G., Ravalico, D., Rovito, L., Manzoni, L., & De Lorenzo, A. (2024). Enhancing Large Language Models-Based Code Generation by Leveraging Genetic Improvement. In M. Giacobini, B. Xue, & L. Manzoni (Eds.), Genetic Programming: 27th European Conference, EuroGP 2024, Proceedings (pp. 108–124). Springer Nature Switzerland AG. Available at: https://link.springer.com/chapter/10.1007/978-3-031-56957-9_7. Accessed on 14. 10. 2024.

- Pryzant, R., Sap, M., Paullada, A., Mishra, S., Puri, R., & Choi, Y. (2023) 'Automatic Prompt Optimization with "Gradient Descent" and Beam Search'. Available at: http://arxiv.org/abs/2305.03495. Accessed on 10. 10. 2024.

- Sabbatella, A., Ponti, A., Candelieri, A., Giordani, I., & Archetti, F. (2023). A Bayesian Approach for Prompt Optimization in Pre-trained Language Models. Available at: https://arxiv.org/abs/2312.00471. Accessed on 16.10.2024

- Sun, H., Hüyük, A. and van der Schaar, M. (2023) 'Query-Dependent Prompt Evaluation and Optimization with Offline Inverse RL'. Available at: http://arxiv.org/abs/2309.06553. Accessed on 10. 10. 2024.

- Yilmaz, M., Clarke, P., & Gormez, M. (2024). Large Language Models for Software Engineering: A Systematic Mapping Study. In Systems, Software and Services Process Improvement (pp. 64–80). Springer Nature Switzerland. Available at: https://doi.org/10.1007/978-3-031-71139-8_5. Accessed on 14.10.2024.

- Zhang, T., Zhao, Z., Deng, M., Liu, K., Sun, M., & Zhang, C. (2022) 'TEMPERA: Test-Time Prompting via Reinforcement Learning'. Available at: http://arxiv.org/abs/2211.11890. Accessed on 11. 10. 2024.